RecurringAlignment: Lexical Drift Detection and In-Context Realignment for AI-Assistant Conversations

Benjamin Frohman (frohmanbenjamin@gmail.com)
October 2026 · Technical report · v0.2.0

Abstract

Long conversations with AI assistants tend to *drift*. The assistant loses track of the user's goal, breaks constraints stated earlier, reverses decisions already made, or adopts a different persona. Restarting the session throws away context and costs tokens. Continuing without correction compounds the error. We describe **RecurringAlignment**, an open-source TypeScript toolkit that (i) scores each assistant turn against a user-defined *anchor* using cheap, deterministic lexical heuristics; (ii) emits a compact *realignment prompt* to be injected into the same context; and (iii) offers *Drift Replay*, which finds where drift began and builds a smaller clean state from which a fresh session can resume. The method needs no API keys or network access. An LLM judge can be plugged in optionally. This report describes the method and its implementation and is candid about its limitations: the signals are lexical rather than semantic, the system has not yet been evaluated on real conversations, and token savings on short chats are small.

1. Introduction

A user working with an assistant over many turns builds up implicit state: what they are trying to do, what they have ruled out, and what has been settled. Language models track this state imperfectly, especially as context grows or the conversation branches. Typical failures include *goal drift* (answering a different question), *constraint violation* ("never use Python" followed later by Python code), *decision reversal* (re-opening a settled choice), *persona drift*, and *topic drift*.

RecurringAlignment makes that implicit state explicit as an **anchor** and checks every assistant turn against it. The design goals are: (1) zero cost and determinism by default, so the check can run on every turn; (2) explanations a user can read; (3) corrections that fit within the same context window instead of requiring a restart; and (4) a principled way to restart when correction is no longer worthwhile.

2. Method

2.1 Anchors

An anchor is a tuple $A = (G, C, P, D)$:

- **goals** G: free text
- **constraints** C: either strings or structured rules $\{\text{rule}, \text{forbid}[\,], \text{pattern}\}$
- an optional **persona** P
- **decisions** D

Constraints written as negations ("never use X", "avoid Y") are compiled automatically into forbidden-token lists. Anchors are stored in a versioned JSON state file, written atomically, so they persist across sessions.

Decision tracking. User turns matching patterns such as "we decided (to) …", "decision: …", or "let's go with …" are appended to D during the session. The monitor emits a typed `decision` event when this happens.

2.2 Text representation

Text is lowercased and stripped of punctuation. Stop-words and tokens of two characters or fewer are removed, and light suffix stemming is applied. For a small ad-hoc corpus $\mathcal{D}$ of N documents (the reply, the goals, and the context window), term t in document d receives the weight

$$w_{t,d} = \frac{\mathrm{count}(t, d)}{|d|} \cdot \left(\ln \frac{1 + N}{1 + \mathrm{df}(t)} + 1 \right),$$

and documents are compared by cosine similarity

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\sum_t a_t b_t}{\|\mathbf{a}\| \, \|\mathbf{b}\|}.$$

2.3 Drift signals

For an assistant reply R, with goal tokens G and a rolling window W of the last k (default 6) **non-drifted** turns, each of the following signals produces $s_j \in [0, 1]$:

Signal	Definition
goal	$g = \max(\mathrm{coverage}(G, R), \cos(R, G))$; $s = 1 - g / g_{\min}$ if $g < g_{\min}$ (0.2)
topic	$\tau = \cos(R, W \cup G)$; $s = 1 - \tau / \tau_{\min}$ if $\tau < \tau_{\min}$ (0.08)
constraint	$s = 1$ if a forbidden token, phrase, or pattern occurs
decision	reversal cue ("actually", "instead", "switch to") with overlap $o \geq 0.3$: $s = \min(1, 0.5 + o)$; negation with $o \geq 0.5$: $s = 0.8o$
persona	$s = 1$ if the reply self-identifies under a different name
judge	optional external score (e.g. an LLM), $s \in [0, 1]$

Drifted turns are left out of W. Without that exclusion, a run of off-topic replies becomes its own context and suppresses the topic signal on later drifted turns. We observed exactly that failure during development.

2.4 Weighted score and severity

$$\mathrm{score}(R) = \min\left(1, \sum_j \omega_j s_j\right),$$

The default weights are $\omega =$
$(\text{goal } 0.3,\ \text{constraint } 0.6,\ \text{decision } 0.5,\ \text{persona } 0.4,\ \text{topic } 0.3,\ \text{judge } 1.0)$. A turn is *drifted*

when its score is at least the threshold θ (default 0.5). Severity is `none`/`low`/`medium`/`high` with cut-offs at 0.2, 0.5, and 0.8. Each contributing signal is reported with a short reason.

Worked example. Take the goal "build a TypeScript drift detection package" and the context "Let's build the drift detection package". The reply "The detection step can wait; first let me explain how compilers parse source files into syntax trees" covers one of the five goal tokens (coverage 0.2), so the goal signal does not fire. TF-IDF cosine to the context is 0.071, giving $s_{\text{topic}} = 0.113$ and a score of 0.034 (severity `none`). A reply offering a Python script for football scores, under the rule "never use python", scores $\min(1, 0.3 + 0.6 + 0.3) = 1.0$ (severity `high`). The full computation is in `docs/scoring.md`.

2.5 Realignment prompts

On drift, the monitor generates a message beginning `[REALIGN]`. It restates only the anchor components relevant to the reasons that fired (the goal is always included), names the drifted turn and its reasons, and instructs the assistant to continue without mentioning the note. The message is capped at 600 characters by default. The point is to fix the conversation *in place* at a small, bounded cost in tokens rather than paying to rebuild the context.

2.6 Middleware

`withAlignment(chatFn, {anchor})` wraps any chat-completion function. It records new user turns, scores each reply, and when a reply drifts, inserts the realignment note into the **next** request, either as a system message immediately before the latest user message or as a suffix on that user message. Reply text is extracted from plain strings and from OpenAI- and Anthropic-style responses.

2.7 Optional LLM judges

Adapters for OpenAI-compatible (`/chat/completions`) and Anthropic-compatible (`/v1/messages`) endpoints send the anchor, recent context, and reply, and they parse a JSON `{score, reasons}` response. Keys are read only from the environment or the constructor. Judges *fail open*: on error or timeout, the turn is still scored heuristically and the failure is recorded as a zero-weight reason.

2.8 Drift Replay

Given a transcript and an anchor, Replay:

1. Scores all turns.
2. Locates the *drift start*: for each drifted assistant turn, it walks back over immediately preceding assistant turns whose scores are at or above the `low` cut-off.
3. Drops the drifted runs and noise turns ("ok", "thanks").
4. Keeps decisions from D unless a later user turn reverses them (a reversal cue plus at least 50% token overlap).
5. Extracts up to six *key facts*: sentences from retained, non-recent turns ranked by $\cos(\text{sentence}, G)$, de-duplicated at 70% token overlap.
6. Collects *open tasks*: user requests with no on-track assistant reply after them.
7. Keeps the last four retained turns verbatim.

The output is a system prompt plus recent turns, written as Markdown or as JSON that the checker can consume directly. Token counts before and after are estimated as $\lceil \mathrm{chars}/4 \rceil$.

3. Implementation

The toolkit is an npm-workspaces monorepo in strict TypeScript. The packages are `core` (no runtime dependencies), `judges`, `middleware`, `replay`, and a CLI with the commands `check`, `watch`, `report`, `replay`, and `anchor`. The CLI returns exit code 2 when drift is found, so it can be used in scripts. The test suite has 58 `node:test` cases with 99.8% line coverage and 98.2% branch coverage. Heuristic scoring of a turn takes well under a millisecond on commodity hardware.

4. Limitations

- **Lexical, not semantic.** Every signal is computed on surface tokens. Paraphrases ("JS" for "JavaScript"), synonyms, and implicit references go undetected, so recall is limited. Shared vocabulary can also mask a real change in intent, so precision is limited too. Negation handling is pattern-based and brittle. During development, "drop bags in the West Loop" was misread as reversing a decision until the cue list was narrowed.
- **No evaluation on real conversations yet.** The thresholds and weights are hand-set defaults validated only on synthetic examples and unit tests. We report no precision, recall, or user-outcome measurements, and none should be inferred.
- **Small savings on short chats.** Drift Replay's clean state has a roughly fixed overhead. On a 10-turn example it reduced the estimated tokens from 412 to 359 (−13%). On very short chats the clean state can be *larger* than the transcript. The savings only matter for long sessions.
- **English-centric** tokenization and stop-words. Token estimates are rough.
- **Realignment efficacy is untested.** We have not measured whether injected notes actually change model behaviour, or by how much, across models.

5. Future work

1. Build a labelled drift benchmark from real (consented, anonymised) multi-turn conversations, and report per-signal precision and recall plus calibration of the weights and thresholds.
2. Add an optional semantic layer using local sentence embeddings for the goal and topic signals, and possibly NLI-based contradiction detection for decisions.
3. Run A/B studies measuring whether in-context realignment restores task success and how its cost compares with restarting.
4. Learn weights per user or domain from accept/reject feedback on flagged turns.
5. Use tokenizer-accurate cost accounting and abstractive summarisation in Replay.
6. Support multilingual tokenization.

6. Availability

Source code under the MIT license: https://github.com/BenFrohman/RecurringAlignment (v0.2.0).